

LAB 4 - Mandelbroot

A.1 Individual results for each parallel strategy

Original Tile

- Prova Seqüencial: Compilació + Execució

Shell

```
paco1114@boada-11:~/lab4$ ./mandel-seq-iter -i 1000 -s 3.0 -c -0.5 0.0 -o -h
Computation of the Mandelbrot set with:
  center = (-0.5, 0)
  size = 3
  maximum iterations = 1000
Total execution time (in seconds): 0.201626
Mandelbrot set: Computed
Histogram for Mandelbrot set: Computed
Writing output file to disk: mandel_image.jpg
```

- Prova OpenMP:

Shell

```
paco1114@boada-12:~/lab4$ ./mandel-omp-iter -i 1000 -s 3.0 -c -0.5 0.0 -o -h
Computation of the Mandelbrot set with:
  center = (-0.5, 0)
  size = 3
  maximum iterations = 1000
Total execution time (in seconds): 0.106613
Mandelbrot set: Computed
Histogram for Mandelbrot set: Computed
Writing output file to disk: mandel_image.jpg
```

→ Comparem:

Shell

```
paco1114@boada-11:~/lab4$ cmp reference_histogram.out mandel_histogram.out
paco1114@boada-11:~/lab4$ cmp reference_image.jpg mandel_image.jpg
```

Tot OK perquè no surt res de output, són iguals els histogrames i les imatges, obtenim menys temps d'execució amb la versió OpenMP: 0.20016 s (seq) > 0.106613 s (OMP).

Proves amb assignació de threads: 1 vs 4 → SpeedUp = 0.1942/0.114670 = 1.694

Shell

```
paco1114@boada-12:~/lab4$ export OMP_NUM_THREADS=1
paco1114@boada-12:~/lab4$ ./mandel-omp-iter -i 1000 -s 3.0 -c -0.5 0.0 -o -h
Computation of the Mandelbrot set with:
    center = (-0.5, 0)
    size = 3
    maximum iterations = 1000
Total execution time (in seconds): 0.194224
Mandelbrot set: Computed
Histogram for Mandelbrot set: Computed
Writing output file to disk: mandel_image.jpg
paco1114@boada-12:~/lab4$ export OMP_NUM_THREADS=4
paco1114@boada-12:~/lab4$ ./mandel-omp-iter -i 1000 -s 3.0 -c -0.5 0.0 -o -h
Computation of the Mandelbrot set with:
    center = (-0.5, 0)
    size = 3
    maximum iterations = 1000
Total execution time (in seconds): 0.114670
Mandelbrot set: Computed
Histogram for Mandelbrot set: Computed
Writing output file to disk: mandel_image.jpg
```

- 2.2.Execució amb sbatch + 2.3.Correctness

Shell

```
paco1114@boada-11:~/lab4$ sbatch submit-seq.sh mandel-seq-iter -h -o -i 10000
Submitted batch job 352321

paco1114@boada-11:~/lab4$ mv mandel_histogram.out seq_histogram.out
paco1114@boada-11:~/lab4$ mv mandel_image.jpg seq_image.jpg

paco1114@boada-11:~/lab4$ sbatch submit-omp.sh mandel-omp-iter 20
Submitted batch job 352324

paco1114@boada-11:~/lab4$ cmp seq_histogram.out mandel_histogram.out
paco1114@boada-11:~/lab4$ cmp seq_image.jpg mandel_image.jpg
```

- 2.4.Performance Analysis

Shell

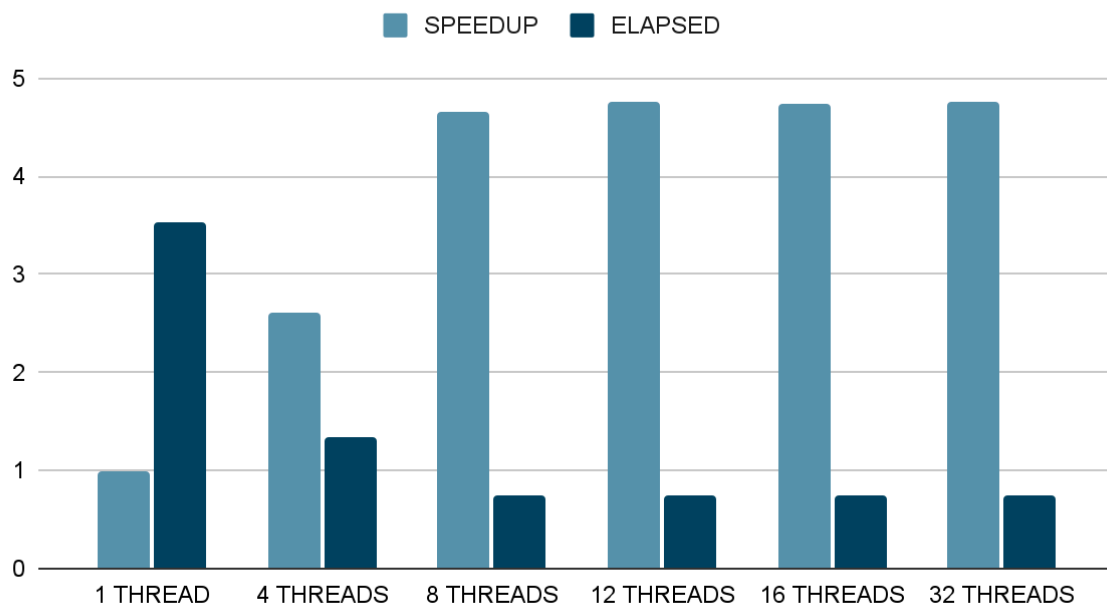
```
paco1114@boada-11:~/lab4$ sbatch submit-strong-omp.sh mandel-omp-iter

paco1114@boada-12:~/lab4$ cat speedup-partial-mandel-omp-iter-boada-19.txt
1    .99214027661163739670
4    2.61790002713412906900
8    4.65354342185814651779
12   4.75076634623969524716
16   4.74530350384353975891
20   4.74360494801023316377
```

```
24  4.74655416163205285195
28  4.74382280792755408588
32  4.75242501664492451594
```

```
paco1114@boada-12:~/lab4$ cat elapsed-partial-mandel-omp-iter-boada-19.txt
1   3.539692
4   1.341484
8   0.754666
12  0.739222
16  0.740073
20  0.740338
24  0.739878
28  0.740304
32  0.738964
```

Punts obtinguts



L'escalabilitat de la versió iterativa “original tile” és bona fins a aproximadament **8–12 threads**, però es **satura cap als 4.7 (16 threads aprox)**.

Això es deu a:

- contenció pel vector d'histograma (atòmics),
- granularitat massa grossa (una tasca per tile),
- overhead de creació i sincronització de tasques,
- i manca d'equilibri de càrrega dins de cada tile.

Per superar aquesta limitació, cal aplicar l'estratègia de **fine-grain tile decomposition**, que incrementa el paral·lisme intern i redueix contencions.

Modelfactor analysis

Overview of whole program execution metrics										
Number of threads	1	2	4	8	12	16	20	24	28	32
Elapsed time (sec)	3.43	1.93	1.01	0.73	0.72	0.72	0.72	0.72	0.72	0.72
Speedup	1.00	1.78	3.39	4.68	4.78	4.78	4.77	4.78	4.78	4.78
Efficiency	1.00	0.89	0.85	0.58	0.40	0.30	0.24	0.20	0.17	0.15

Table 1: Analysis done on Wed Nov 26 10:15:37 AM CET 2025, paco1114

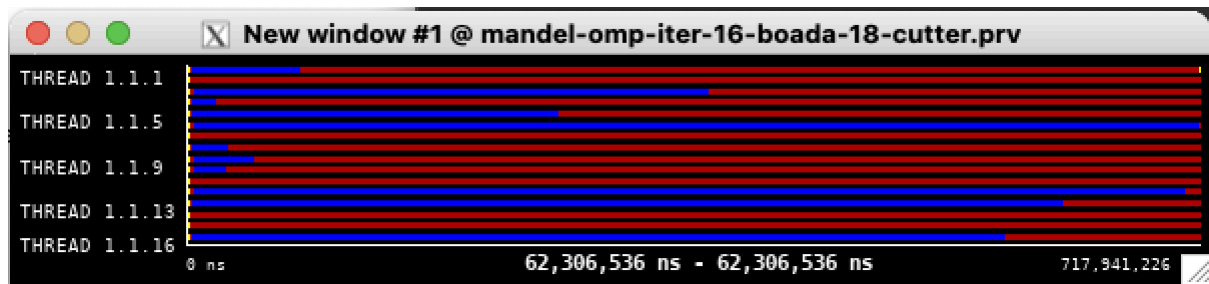
Overview of the Efficiency metrics in parallel fraction, $\phi=99.99\%$										
Number of threads	1	2	4	8	12	16	20	24	28	32
Global efficiency	100.00%	88.78%	84.88%	58.51%	39.85%	29.87%	23.86%	19.93%	17.08%	14.95%
Parallelization strategy efficiency	100.00%	88.86%	85.01%	58.74%	40.10%	30.22%	24.18%	20.23%	17.41%	15.21%
Load balancing	100.00%	88.89%	85.05%	58.79%	40.13%	30.25%	24.24%	20.26%	17.46%	15.24%
In execution efficiency	100.00%	99.97%	99.95%	99.90%	99.91%	99.89%	99.77%	99.84%	99.73%	99.81%
Scalability for computation tasks	100.00%	99.91%	99.85%	99.62%	99.39%	98.84%	98.67%	98.50%	98.08%	98.30%
IPC scalability	100.00%	99.99%	99.97%	99.97%	99.96%	99.95%	99.95%	99.94%	99.95%	99.96%
Instruction scalability	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
Frequency scalability	100.00%	99.92%	99.88%	99.65%	99.44%	98.89%	98.72%	98.55%	98.13%	98.34%

Table 2: Analysis done on Wed Nov 26 10:15:37 AM CET 2025, paco1114

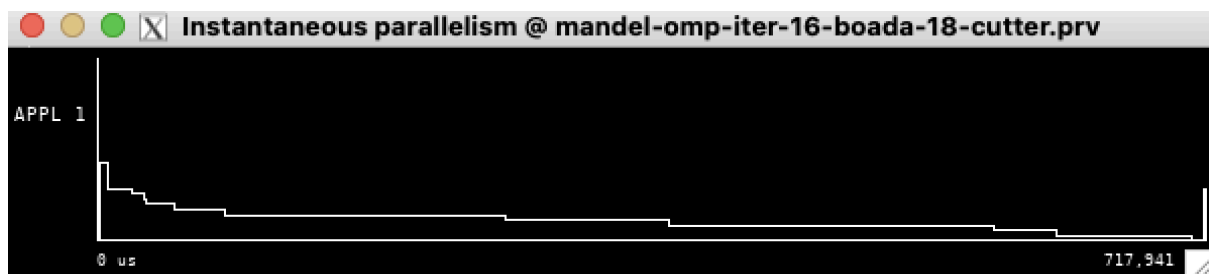
Statistics about explicit tasks in parallel fraction										
Number of threads	1	2	4	8	12	16	20	24	28	32
Number of explicit tasks executed (total)	64.0	64.0	64.0	64.0	64.0	64.0	64.0	64.0	64.0	64.0
LB (number of explicit tasks executed)	1.0	0.94	0.64	0.44	0.31	0.24	0.19	0.16	0.13	0.12
LB (time executing explicit tasks)	1.0	0.89	0.85	0.59	0.4	0.3	0.24	0.2	0.17	0.15
Time per explicit task (average us)	53589.47	53634.4	53653.64	53769.47	53853.1	54095.18	54123.95	54161.3	54219.98	53883.49
Overhead per explicit task (synch %)	0.0	12.52	17.59	70.13	149.36	231.07	313.64	395.3	476.48	562.9
Overhead per explicit task (sched %)	0.0	0.01	0.0	0.0	0.0	0.0	0.0	0.01	0.0	0.0
Number of taskwait/taskgroup (total)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

Table 3: Analysis done on Wed Nov 26 10:15:37 AM CET 2025, paco1114

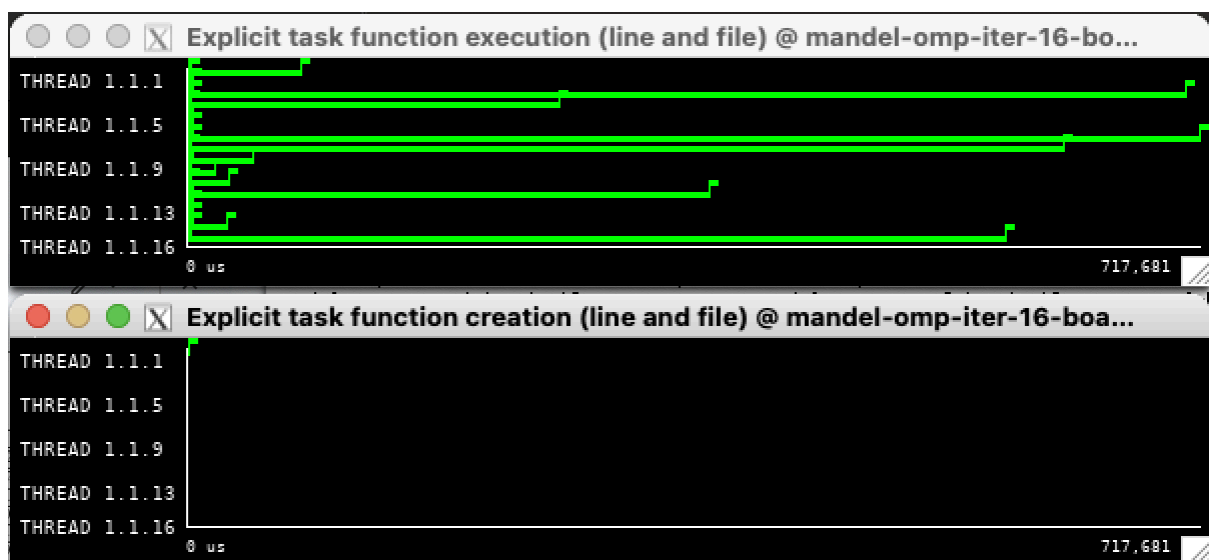
Paraver analysis



Aquesta vista mostra clarament un **greu desequilibri de càrrega**. Les barres blaves representen el temps de còmput útil i les vermelles el temps d'espera. Observem l'anomenat 'efecte escala': com que els tiles tenen costos molt diferents i són massa grans, alguns fils acaben molt ràpid i es queden esperant (vermell) fins que el fil més lent acaba la seva tasca, desaprofitant molta CPU.



Aquesta gràfica confirma la pèrdua d'eficiència temporal. La línia comença al màxim (tots els fils treballant), però cau progressivament formant esglaons. Això demostra que, a mesura que avança l'execució, el grau de paral·lelisme disminueix perquè cada cop hi ha menys fils treballant simultàniament.



Aquí veiem la durada real de cada tasca. Les línies verdes tenen longituds extremadament disperses, la qual cosa confirma que la **granularitat és massa grossa**. Les tasques que cauen dins del conjunt de Mandelbrot triguen molt més a calcular-se que les de fora, provocant el desequilibri vist anteriorment.

Fine-grain

- Prova OpenMP:

Shell

```
paco1114@boada-12:~/lab4$ ./mandel-omp-iter -i 1000 -s 3.0 -c -0.5 0.0 -o -h
Computation of the Mandelbrot set with:
  center = (-0.5, 0)
  size = 3
  maximum iterations = 1000
Total execution time (in seconds): 0.108409
Mandelbrot set: Computed
Histogram for Mandelbrot set: Computed
Writing output file to disk: mandel_image.jpg
```

- 2.2.Execució amb sbatch + 2.3.Correctness

Shell

```
paco1114@boada-11:~/lab4$ sbatch submit-omp.sh mandel-omp-iter 20
Submitted batch job 352439

paco1114@boada-11:~/lab4$ cmp seq_histogram.out mandel_histogram.out
paco1114@boada-11:~/lab4$ cmp seq_image.jpg mandel_image.jpg

paco1114@boada-12:~/lab4$ mv mandel_histogram.out finegrain_tile_hist.out
paco1114@boada-12:~/lab4$ mv mandel_image.jpg finegrain_tile_img.jpg
```

- 2.4.Performance Analysis

Shell

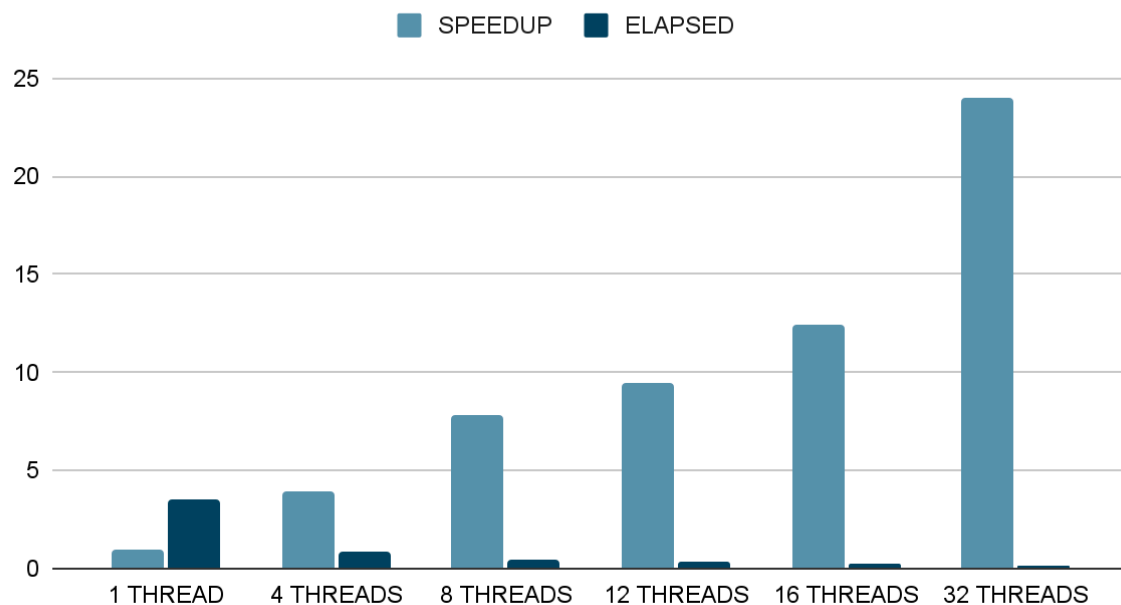
```
paco1114@boada-12:~/lab4$ sbatch submit-strong-omp.sh mandel-omp-iter
Submitted batch job 358049

paco1114@boada-12:~/lab4$ cat speedup-partial-mandel-omp-iter-boada-18.txt
1 .99266792495605079931
4 3.94936975073439109643
8 7.79873502899262026218
12 9.44830119029682085279
16 12.44328417442082958322
20 15.34176656647560464140
24 18.34181565573470805455
28 21.19421208280523869877
32 23.99566096784333019009

paco1114@boada-12:~/lab4$ cat elapsed-partial-mandel-omp-iter-boada-18.txt
1 3.537607
4 0.889172
8 0.450287
```

12 0.371672
16 0.282214
20 0.228896
24 0.191457
28 0.165690
32 0.146346

Punts obtinguts



En la versió iterativa “original tile”, el speedup es saturava al voltant de $S \approx 4.7$ a partir de 12 fils, tot i augmentar el nombre de threads fins a 32. Aquesta saturació es deu a la granularitat massa grossa (una tasca per tile amb molta feina seqüencial a dins) i a la contenció sobre el vector d'histograma.

En canvi, en la versió **fine-grain**, on es descompon el càlcul del tile en tasques més petites (una tasca per fila en el cas de càlcul píxel a píxel), el speedup millora de forma significativa: obtenim $S(16) \approx 12.44$ i $S(32) \approx 23.99$, amb eficiències pròximes al **90%** fins i tot amb 32 fils. Això mostra que la descomposició fine-grain explota molt millor el paral·lelisme disponible i redueix el temps idle dels threads.

Modelfactor analysis

Overview of whole program execution metrics										
Number of threads	1	2	4	8	12	16	20	24	28	32
Elapsed time (sec)	4.22	2.12	1.07	0.54	0.36	0.28	0.23	0.20	0.17	0.16
Speedup	1.00	1.99	3.95	7.79	11.56	14.94	18.32	21.20	24.18	27.15
Efficiency	1.00	0.99	0.99	0.97	0.96	0.93	0.92	0.88	0.86	0.85

Table 1: Analysis done on Wed Nov 26 10:28:26 AM CET 2025, paco1114

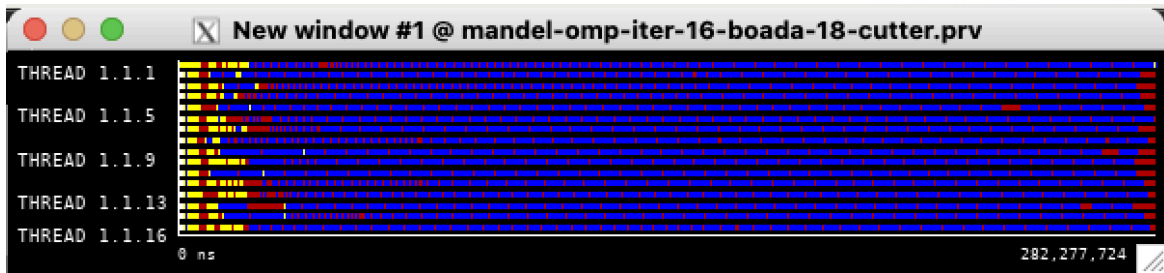
Overview of the Efficiency metrics in parallel fraction, $\phi=99.99\%$										
Number of threads	1	2	4	8	12	16	20	24	28	32
Global efficiency	99.95%	99.44%	98.88%	97.40%	96.33%	93.43%	91.76%	88.49%	86.46%	85.01%
Parallelization strategy efficiency	99.95%	99.57%	99.04%	97.62%	96.64%	94.12%	92.46%	89.47%	87.46%	86.45%
Load balancing	100.00%	99.99%	99.68%	99.24%	98.68%	97.37%	97.17%	94.58%	91.24%	90.63%
In execution efficiency	99.95%	99.58%	99.35%	98.37%	97.94%	96.66%	95.15%	94.60%	95.86%	95.38%
Scalability for computation tasks	100.00%	99.87%	99.84%	99.77%	99.68%	99.27%	99.25%	98.90%	98.86%	98.33%
IPC scalability	100.00%	99.89%	99.87%	99.87%	99.87%	99.83%	99.82%	99.81%	99.82%	99.82%
Instruction scalability	100.00%	100.08%	100.08%	100.08%	100.08%	100.08%	100.08%	100.08%	100.08%	100.08%
Frequency scalability	100.00%	99.90%	99.89%	99.82%	99.73%	99.36%	99.35%	99.01%	98.96%	98.43%

Table 2: Analysis done on Wed Nov 26 10:28:26 AM CET 2025, paco1114

Statistics about explicit tasks in parallel fraction										
Number of threads	1	2	4	8	12	16	20	24	28	32
Number of explicit tasks executed (total)	8384.0	8384.0	8384.0	8384.0	8384.0	8384.0	8384.0	8384.0	8384.0	8384.0
LB (number of explicit tasks executed)	1.0	0.96	0.63	0.32	0.31	0.4	0.43	0.43	0.5	0.45
LB (time executing explicit tasks)	1.0	1.0	1.0	0.99	0.99	0.97	0.97	0.95	0.91	0.91
Time per explicit task (average us)	502.29	503.05	503.19	503.39	503.38	505.47	504.98	505.81	505.46	506.68
Overhead per explicit task (synch %)	0.0	0.28	0.66	1.59	2.03	3.73	4.99	6.63	7.67	8.63
Overhead per explicit task (sched %)	0.04	0.14	0.29	0.69	1.17	2.23	2.7	4.4	5.82	6.16
Number of taskwait/taskgroup (total)	128.0	128.0	128.0	128.0	128.0	128.0	128.0	128.0	128.0	128.0

Table 3: Analysis done on Wed Nov 26 10:28:26 AM CET 2025, paco1114

Paraver analysis



Aquesta vista mostra una millora dràstica en el **balanceig de càrrega** respecte a la versió Original Tile. En lloc de blocs llargs i desiguals, veiem una distribució molt més granular i uniforme de la feina. El més important és observar la part final (a la dreta): pràcticament no hi ha espais vermells i tots els fils acaben la seva execució gairebé al mateix temps, aprofitant al màxim els recursos.



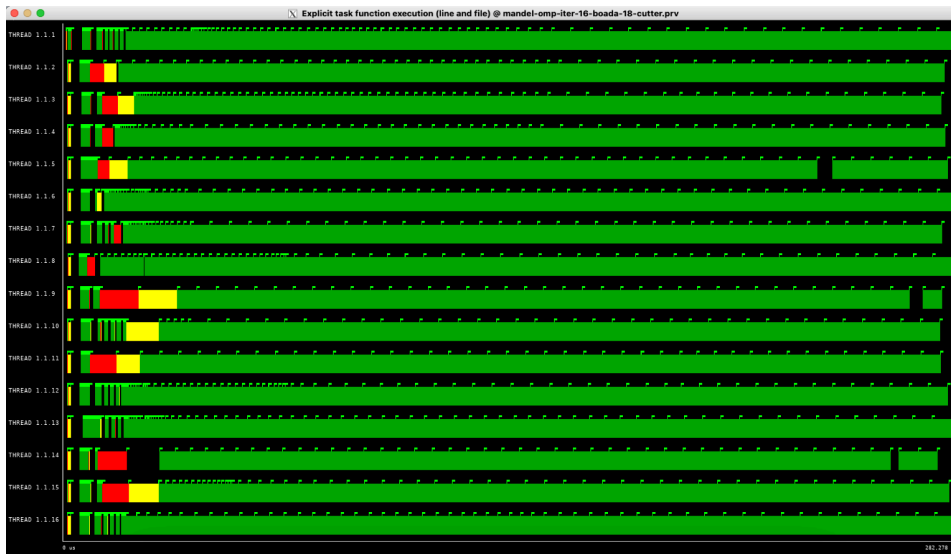
Aquesta és la gràfica ideal. La línia de paral·lelisme es dispara al màxim a l'inici i es **manté plana a dalt de tot** durant tota l'execució, sense caigudes ni esglaons. Això demostra una ocupació del 100% de la CPU, confirmant que l'estratègia de gra fi ha eliminat els temps morts que patia la versió de blocs grans.

- Creation



Mostra l'instant de creació de les tasques. En aquesta estratègia, es generen més tasques que en l'anterior (una per fila en lloc d'una per tile gran). Tot i que hi ha més volum de creació, el sistema ho gestiona ràpidament a l'inici, permetent que l'execució comenci de seguida sense introduir un *overhead* excessiu.

- Execution



Aquí es visualitza la **granularitat fina**. Les barres verdes (execució de tasques) són molt més curtes i nombroses. Com que les tasques són petites, l'aplanador (*scheduler*) d'OpenMP pot assignar feina nova ràpidament als fils que queden lliures, omplint els forats i aconseguint el balanceig perfecte que hem vist a la primera gràfica.

Leaf strategy

3.2.Execution + 3.3.Correctness

None

```
paco1114@boada-12:~/lab4$ make mandel-seq-rec
icx -I. -I/Soft/stb/20200430 -o mandel-seq-rec -g -O3 -fp-model precise -std=c99
mandel-seq-rec.c -lX11 -L/usr/X11R6/lib -lm
paco1114@boada-12:~/lab4$ sbatch submit-seq.sh mandel-seq-rec -h -o -i 10000
Submitted batch job 362544
paco1114@boada-12:~/lab4$ squeue -u $USER
```

JOBID	PARTITION	NAME	USER	ST	TIME	NODES
362514	interacti	_interac	paco1114	R	25:47	1 boada-12
362512	interacti	_interac	paco1114	R	27:23	1 boada-12

```
paco1114@boada-12:~/lab4$ mv mandel_histogram.out mandel_histogram_seq_rec.out
paco1114@boada-12:~/lab4$ mv mandel_image.jpg mandel_image_seq_rec.jpg
```

```
paco1114@boada-12:~/lab4$ make mandel-omp-rec
icx -I. -I/Soft/stb/20200430 -o mandel-omp-rec -g -O3 -fp-model precise -std=c99
-qopenmp mandel-omp-rec.c -lX11 -L/usr/X11R6/lib -lm
paco1114@boada-12:~/lab4$ sbatch submit-omp.sh mandel-omp-rec 20
Submitted batch job 362454
paco1114@boada-12:~/lab4$ mv mandel_histogram.out mandel_histogram_leaf_rec.out
paco1114@boada-12:~/lab4$ mv mandel_image.jpg mandel_image_leaf_rec.jpg
```

```
paco1114@boada-12:~/lab4$ cmp mandel_histogram_seq_rec.out  
mandel_histogram_leaf_rec.out  
paco1114@boada-12:~/lab4$ cmp mandel_image_seq_rec.jpg mandel_image_leaf_rec.jpg
```

2.4. Performance analysis

Aquesta estratègia presenta un rendiment molt deficient, amb un Speedup que es satura en 1.29 independentment del nombre de fils. Això es deu a una granularitat excessivament fina: en crear una tasca (task) per a cada crida recursiva fins al nivell de píxel, es generen milers de tasques minúscules.

La taula d'estadístiques confirma aquest coll d'ampolla mostrant un overhead de sincronització desorbitat (superant el 10.000% amb 32 fils). El temps que la CPU perd gestionant la creació i planificació d'aquestes tasques anul·la qualsevol benefici de l'execució en paral·lel.

Modelfactor analysis

Overview of whole program execution metrics										
Number of threads	1	2	4	8	12	16	20	24	28	32
Elapsed time (sec)	1.83	1.42	1.42	1.42	1.42	1.42	1.42	1.42	1.42	1.42
Speedup	1.00	1.29	1.29	1.29	1.29	1.29	1.29	1.29	1.29	1.29
Efficiency	1.00	0.64	0.32	0.16	0.11	0.08	0.06	0.05	0.05	0.04

Table 1: Analysis done on Wed Dec 3 08:53:31 AM CET 2025, paco1114

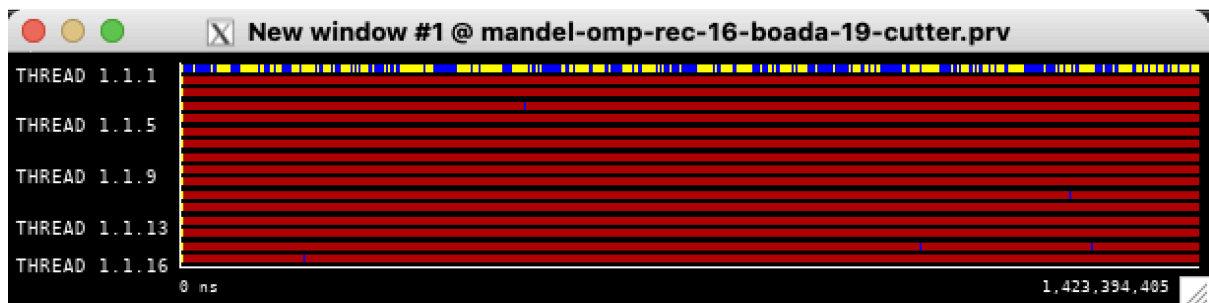
Overview of the Efficiency metrics in parallel fraction, $\phi=99.98\%$										
Number of threads	1	2	4	8	12	16	20	24	28	32
Global efficiency	99.96%	64.42%	32.23%	16.09%	10.73%	8.05%	6.43%	5.36%	4.59%	4.02%
Parallelization strategy efficiency	99.96%	64.55%	32.30%	16.15%	10.78%	8.10%	6.50%	5.43%	4.68%	4.11%
Load balancing	100.00%	64.74%	32.40%	16.21%	10.82%	8.13%	6.52%	5.46%	4.69%	4.12%
In execution efficiency	99.96%	99.70%	99.70%	99.64%	99.68%	99.67%	99.61%	99.59%	99.64%	99.57%
Scalability for computation tasks	100.00%	99.81%	99.77%	99.64%	99.49%	99.29%	98.99%	98.61%	98.25%	97.87%
IPC scalability	100.00%	99.84%	99.79%	99.72%	99.73%	99.71%	99.72%	99.70%	99.71%	99.69%
Instruction scalability	100.00%	100.07%	100.07%	100.06%	100.06%	100.06%	100.06%	100.06%	100.06%	100.06%
Frequency scalability	100.00%	99.91%	99.91%	99.85%	99.70%	99.52%	99.21%	98.85%	98.48%	98.12%

Table 2: Analysis done on Wed Dec 3 08:53:31 AM CET 2025, paco1114

Statistics about explicit tasks in parallel fraction										
Number of threads	1	2	4	8	12	16	20	24	28	32
Number of explicit tasks executed (total)	3031.0	3031.0	3031.0	3031.0	3031.0	3031.0	3031.0	3031.0	3031.0	3031.0
LB (number of explicit tasks executed)	1.0	0.54	0.68	0.92	0.84	0.82	0.8	0.79	0.79	0.75
LB (time executing explicit tasks)	1.0	0.5	0.7	0.74	0.75	0.8	0.66	0.61	0.46	0.45
Time per explicit task (average us)	137.49	137.92	138.21	138.29	138.2	138.29	138.29	138.21	138.24	138.35
Overhead per explicit task (synch %)	0.0	240.35	917.72	2274.43	3633.94	4987.7	6346.14	7705.89	9061.4	10413.08
Overhead per explicit task (sched %)	0.18	0.67	0.77	0.84	0.79	0.78	0.81	0.84	0.79	0.93
Number of taskwait/taskgroup (total)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

Table 3: Analysis done on Wed Dec 3 08:53:31 AM CET 2025, paco1114

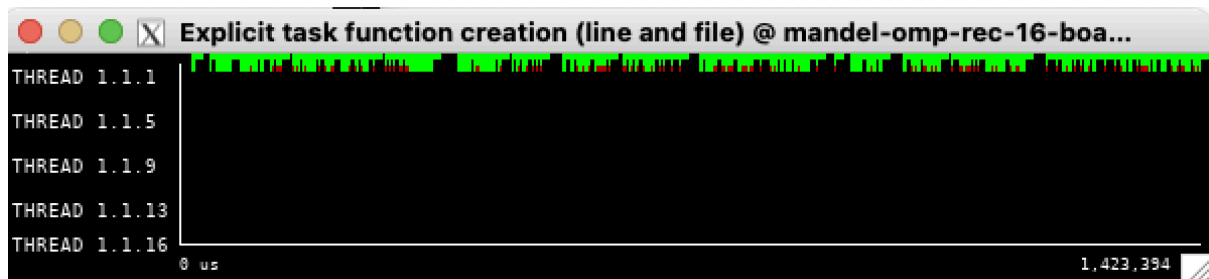
Paraver analysis



Aquesta vista és devastadora: el color vermell (sincronització/overhead) domina completament la traça, mentre que el blau (còmput útil) és gairebé invisible. Això confirma visualment el problema de la granularitat excessiva: els fils passen pràcticament tot el temps gestionant l'estructura de tasques en lloc de calcular el fractal.



El paral·lelisme és extremadament baix i irregular. Després d'un pic inicial (quan es llencen les primeres crides), la línia cau i es manté molt a prop de zero amb petits pics esporàdics. Això indica que la majoria de fils estan bloquejats o gestionant overhead, sense treballar simultàniament en el problema real.



S'observa una **barra verda sòlida i contínua** a la part superior. Això indica una freqüència massiva i constant de creació de tasques. El sistema està saturat creant noves tasques (una per a cada fulla de la recursió) sense descans, la qual cosa és la causa principal de l'overhead.



Les tasques (línies verdes) són extremadament curtes (microscòpiques) i estan molt disperses. Això demostra que la feina útil de cada tasca és tan petita que no compensa el cost de crear-la, confirmant la necessitat urgent d'introduir un *cutoff*.

Strong scalability test

Shell

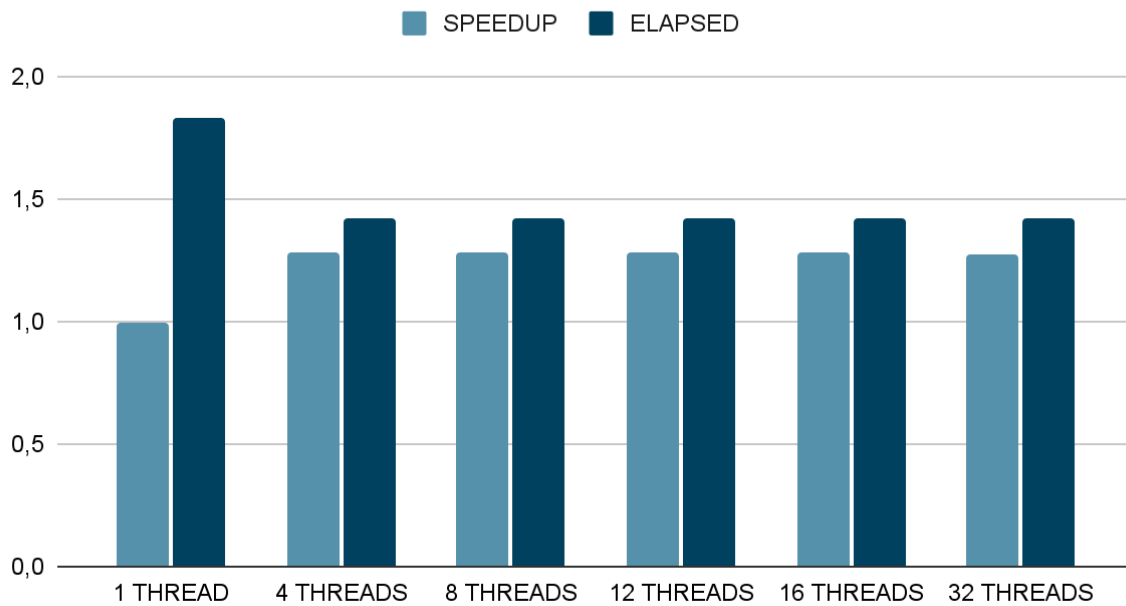
```
paco1114@boada-11:~/lab4$ cat speedup-partial-mandel-omp-rec-boada-19.txt
```

```
1    .99300714597822027025
4    1.27996125845180427341
8    1.27888397670983859301
12   1.27889478558826325965
16   1.27789933430500358570
20   1.27764397113716881097
24   1.27677075357005492284
28   1.27634446090821929748
32   1.27616236369833890312
```

```
paco1114@boada-11:~/lab4$ cat elapsed-partial-mandel-omp-rec-boada-19.txt
```

```
1    1.828581
4    1.418632
8    1.419827
12   1.419815
16   1.420921
20   1.421205
24   1.422177
28   1.422652
32   1.422855
```

Punts obtinguts



CUTOFF strategy

La introducció d'un cutoff (llindar) per aturar la recursivitat paral·lela quan la mida de la regió és petita ha reduït dràsticament l'overhead de gestió de tasques. Això ha permès desbloquejar el rendiment, assolint un speedup màxim de 4.60.

Malgrat la millora, el rendiment es torna a estancar ràpidament. Aquest comportament és similar al de la versió iterativa Original Tile: encara que hem eliminat l'excés de tasques, el programa continua patint un desequilibri de càrrega significatiu.

Modelfactor analysis

Overview of whole program execution metrics										
Number of threads	1	2	4	8	12	16	20	24	28	32
Elapsed time (sec)	1.83	1.00	0.53	0.40	0.40	0.40	0.40	0.40	0.40	0.40
Speedup	1.00	1.82	3.46	4.60	4.60	4.61	4.61	4.60	4.59	4.59
Efficiency	1.00	0.91	0.87	0.58	0.38	0.29	0.23	0.19	0.16	0.14

Table 1: Analysis done on Wed Dec 3 10:12:10 AM CET 2025, pacol1114

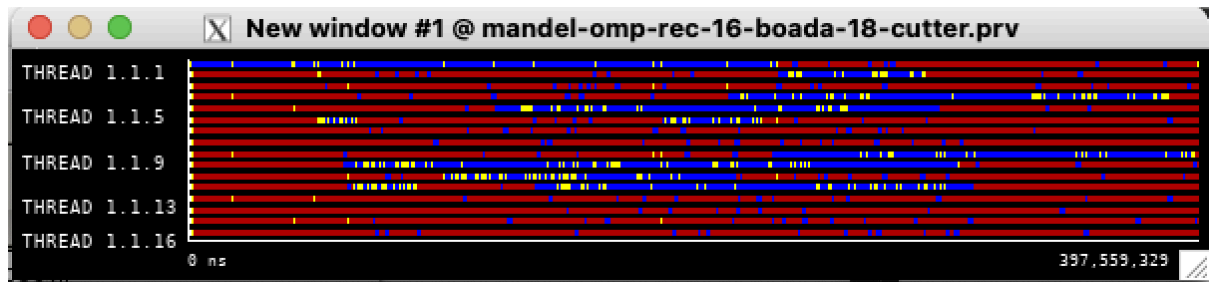
Overview of the Efficiency metrics in parallel fraction, $\phi=99.98\%$										
Number of threads	1	2	4	8	12	16	20	24	28	32
Global efficiency	99.97%	91.16%	86.56%	57.57%	38.37%	28.80%	23.05%	19.19%	16.41%	14.35%
Parallelization strategy efficiency	99.97%	91.29%	86.65%	57.75%	38.53%	28.99%	23.26%	19.42%	16.72%	14.66%
Load balancing	100.00%	95.98%	92.22%	75.50%	59.28%	45.67%	37.15%	30.99%	26.97%	23.80%
In execution efficiency	99.97%	95.12%	93.96%	76.48%	65.00%	63.47%	62.61%	62.67%	61.99%	61.58%
Scalability for computation tasks	100.00%	99.85%	99.89%	99.70%	99.58%	99.37%	99.10%	98.81%	98.15%	97.89%
IPC scalability	100.00%	99.95%	99.96%	99.90%	99.87%	99.84%	99.85%	99.82%	99.82%	99.78%
Instruction scalability	100.00%	100.04%	100.04%	100.04%	100.04%	100.04%	100.04%	100.04%	100.04%	100.04%
Frequency scalability	100.00%	99.86%	99.89%	99.76%	99.67%	99.48%	99.21%	98.95%	98.29%	98.07%

Table 2: Analysis done on Wed Dec 3 10:12:10 AM CET 2025, pacol1114

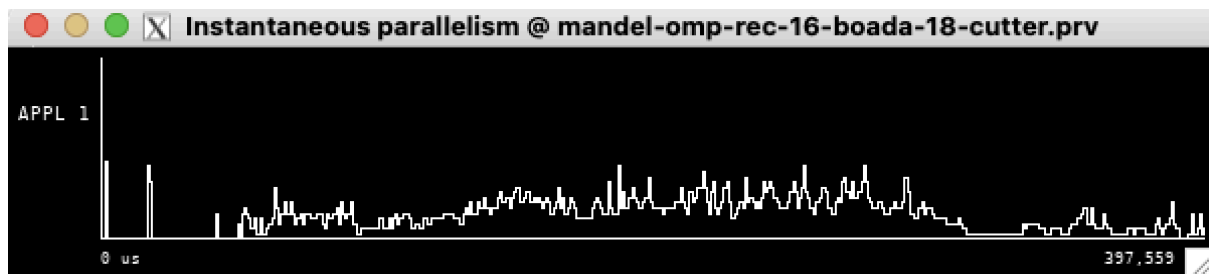
Statistics about explicit tasks in parallel fraction										
Number of threads	1	2	4	8	12	16	20	24	28	32
Number of explicit tasks executed (total)	1854.0	1854.0	1854.0	1854.0	1854.0	1854.0	1854.0	1854.0	1854.0	1854.0
LB (number of explicit tasks executed)	1.0	0.9	0.63	0.5	0.72	0.61	0.71	0.72	0.65	0.62
LB (time executing explicit tasks)	1.0	0.84	0.85	0.66	0.54	0.4	0.33	0.27	0.23	0.21
Time per explicit task (average us)	263.24	864.81	864.58	865.74	865.99	866.47	866.59	866.65	867.24	867.59
Overhead per explicit task (synch %)	0.0	10.81	17.38	83.1	181.42	280.07	378.08	476.69	574.5	672.7
Overhead per explicit task (sched %)	0.09	0.06	0.1	0.12	0.15	0.15	0.18	0.17	0.18	0.19
Number of taskwait/taskgroup (total)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

Table 3: Analysis done on Wed Dec 3 10:12:10 AM CET 2025, pacol1114

Paraver analysis

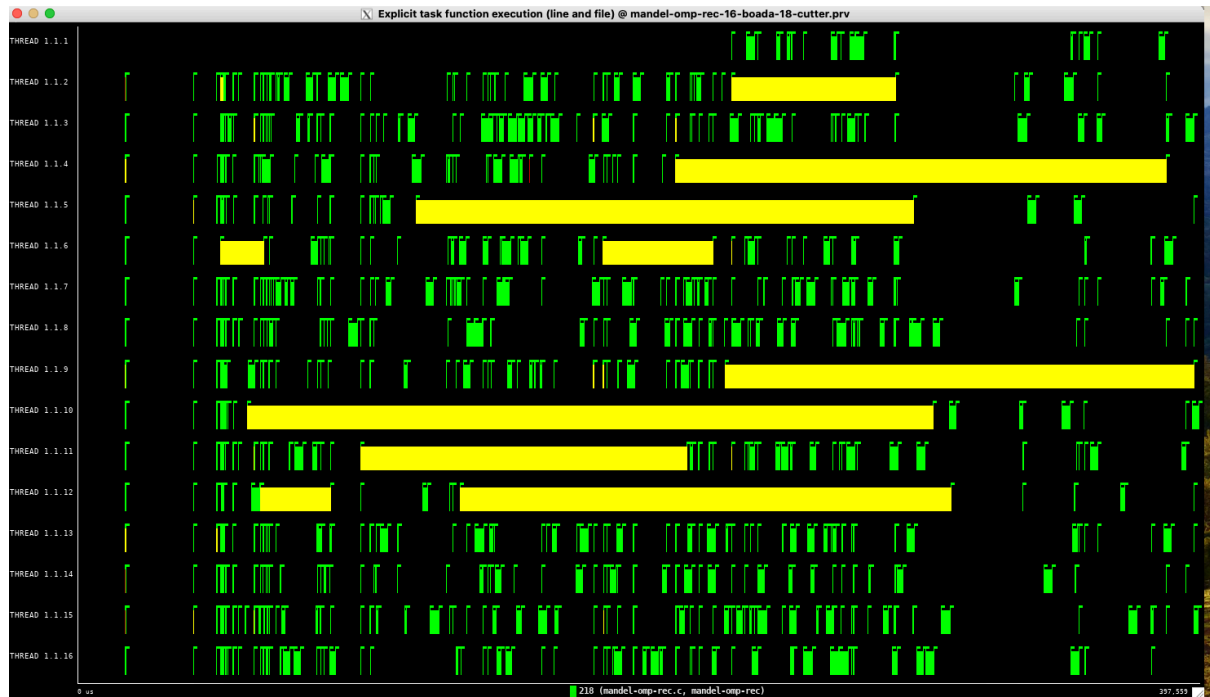


En aquesta vista s'aprecia l'èxit del *cutoff*: la 'marea vermella' (overhead) de la versió anterior ha desaparegut i ara veiem barres blaves de còmput net. No obstant això, reapareix el problema del **desequilibri de càrrega** (l'efecte escala al final): com que aturem la recursivitat aviat, ens quedem amb tasques de mides desiguals, fent que uns fils acabin molt abans que d'altres.



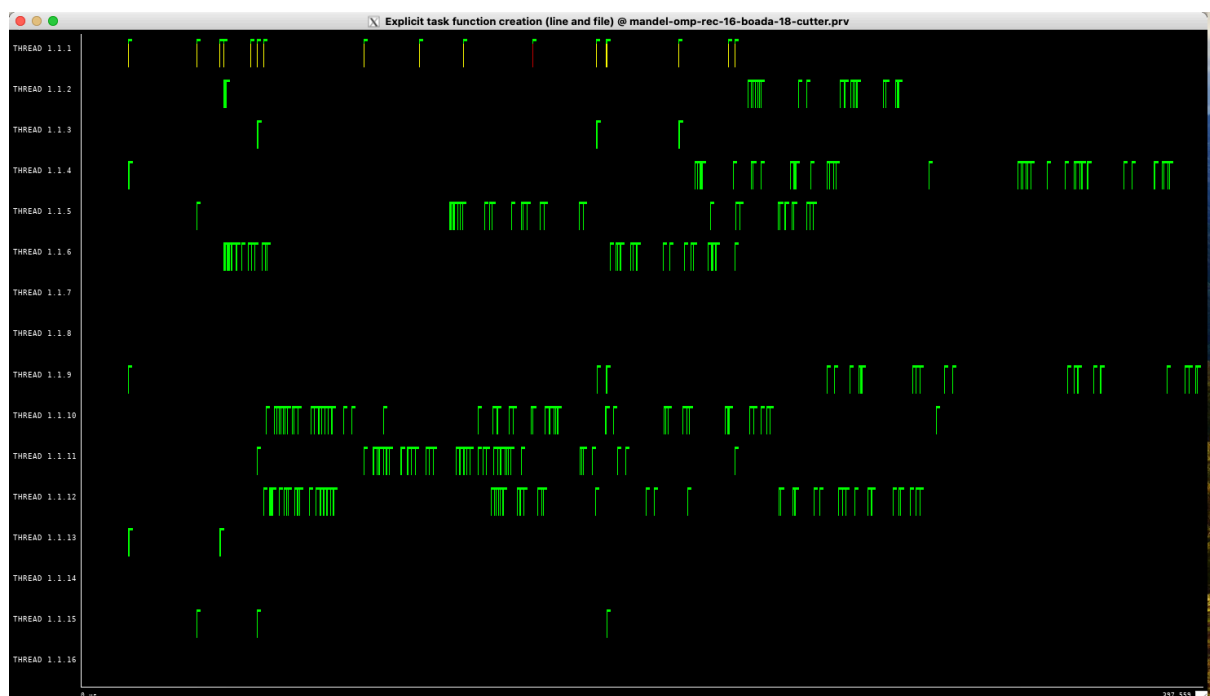
El paral·lisme és molt millor que a la versió *Leaf* pura, mantenint-se alt durant gran part de l'execució. Tot i així, cap al final de la traça la línia cau progressivament (formant esglaons), la qual cosa confirma que la CPU es desaprofita en els instants finals a causa del desequilibri de càrrega.

- Creation



A diferència de la barra verda sòlida de la versió anterior, aquí veiem **línies separades i disperses**. Això demostra visualment que el mecanisme de *cutoff* ha funcionat: s'ha frenat la creació massiva de tasques, generant-ne només la quantitat necessària per distribuir la feina inicialment.

- Execution



Les tasques (barres verdes) ara tenen una **durada visible i significativa**, a diferència dels punts microscòpics de la versió sense *cutoff*. Això indica que la granularitat ha millorat: el temps de còmput de cada tasca és suficientment llarg per compensar el cost de gestionar-la, tot i que la seva durada irregular provoca el desequilibri final.

Strong scalability test

Shell

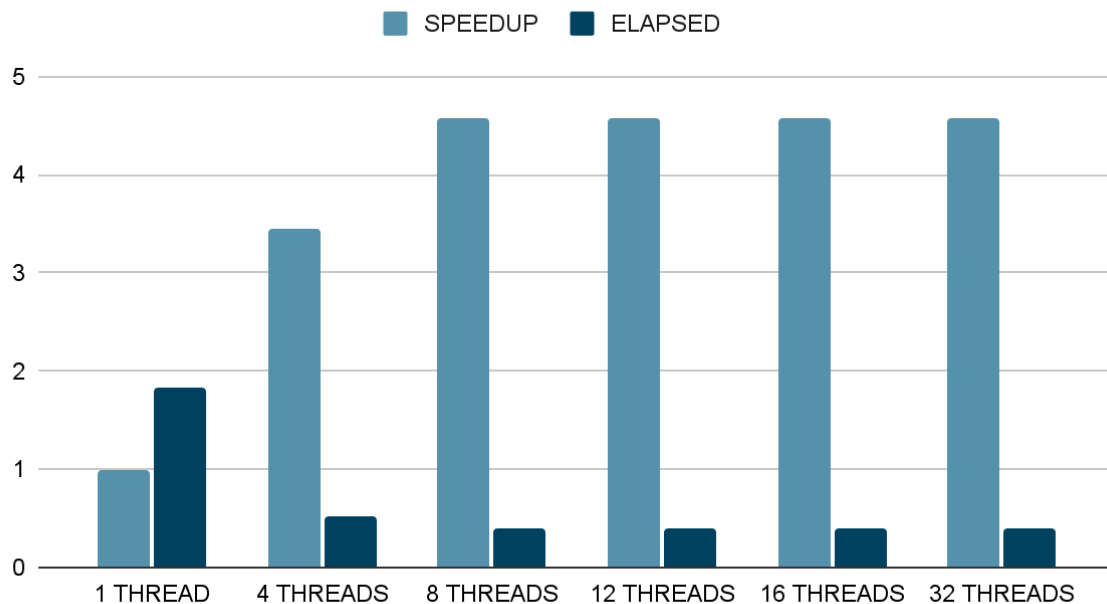
```
paco1114@boada-11:~/lab4$ cat speedup-partial-mandel-omp-rec-boada-20.txt
```

```
1 .99260264297529877516
4 3.44126554546040337650
8 4.57994607287980406647
12 4.57567579145782461551
16 4.57660995589161940768
20 4.57114877248101826677
24 4.56748075051000772243
28 4.57383164857641609003
32 4.56950375849026215060
```

```
paco1114@boada-11:~/lab4$ cat elapsed-partial-mandel-omp-rec-boada-20.txt
```

```
1 1.829302
4 0.527646
8 0.396461
12 0.396831
16 0.396750
20 0.397224
24 0.397543
28 0.396991
32 0.397367
```

Punts obtinguts



Tree strategy

3.2.Execution + 3.3.Correctness

```
Shell
paco1114@boada-11:~/lab4$ make clean
rm -rf mandel-seq-iter mandel-seq-rec mandel-omp-iter mandel-omp-rec logs
*.log
paco1114@boada-11:~/lab4$ make mandel-omp-rec
icx -I. -I/Soft/stb/20200430 -o mandel-omp-rec -g -O3 -fp-model precise
-std=c99 -qopenmp mandel-omp-rec.c -lX11 -L/usr/X11R6/lib -lm
paco1114@boada-11:~/lab4$ sbatch submit-omp.sh mandel-omp-rec 20
Submitted batch job 369928
paco1114@boada-11:~/lab4$ squeue -u $USER
          JOBID PARTITION      NAME      USER ST       TIME  NODES
NODELIST(REASON)
          369922 interacti _interac paco1114  R       6:02      1 boada-11
paco1114@boada-11:~/lab4$ cmp mandel_histogram.out
mandel_histogram_seq_rec.out
paco1114@boada-11:~/lab4$ cmp mandel_image.jpg mandel_image_seq_rec.jpg
```

2.4.Performance analysis

Aquesta és la implementació recursiva amb millor rendiment absolut. El Speedup arriba fins a 17.72 amb 32 fils , i el temps d'execució es redueix a només 0.10 segons.

La millora dràstica respecte a l'estratègia Leaf es deu al fet que paral·lelitzem les branques de l'arbre de recursió en lloc de les fulles finals. Això genera un nombre de tasques molt més òptim i equilibrat, reduint significativament l'overhead de gestió i permetent una escalabilitat molt superior.

Modelfactor analysis

Overview of whole program execution metrics										
Number of threads	1	2	4	8	12	16	20	24	28	32
Elapsed time (sec)	1.84	0.93	0.48	0.26	0.19	0.16	0.13	0.12	0.11	0.10
Speedup	1.00	1.97	3.84	7.03	9.77	11.88	13.77	15.40	16.58	17.72
Efficiency	1.00	0.99	0.96	0.88	0.81	0.74	0.69	0.64	0.59	0.55

Table 1: Analysis done on Thu Dec 4 06:35:09 PM CET 2025, paco1114

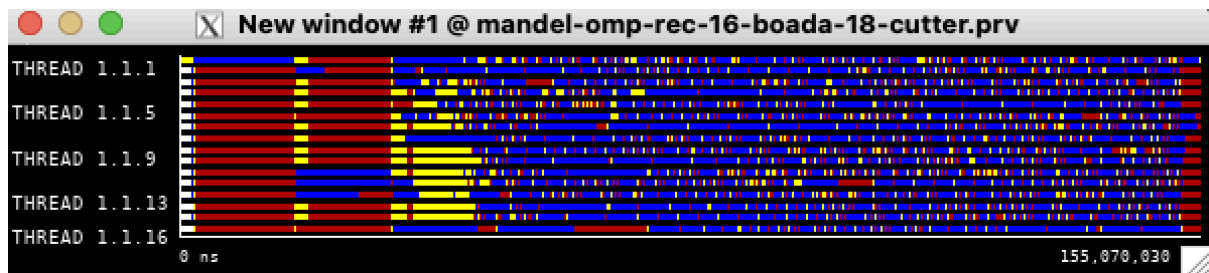
Overview of the Efficiency metrics in parallel fraction, $\phi=99.98\%$										
Number of threads	1	2	4	8	12	16	20	24	28	32
Global efficiency	99.75%	98.34%	95.74%	87.80%	81.32%	74.18%	68.80%	64.16%	59.20%	55.39%
Parallelization strategy efficiency	99.75%	98.37%	95.86%	88.02%	81.72%	74.89%	69.61%	65.10%	60.40%	56.81%
Load balancing	100.00%	99.14%	97.12%	96.81%	90.41%	84.66%	76.24%	75.74%	78.33%	82.33%
In execution efficiency	99.75%	99.22%	98.71%	90.93%	90.39%	88.47%	91.30%	85.96%	77.10%	69.00%
Scalability for computation tasks	100.00%	99.97%	99.87%	99.75%	99.51%	99.05%	98.84%	98.54%	98.02%	97.50%
IPC scalability	100.00%	99.78%	99.72%	99.60%	99.53%	99.36%	99.30%	99.32%	99.29%	99.24%
Instruction scalability	100.00%	100.29%	100.29%	100.29%	100.29%	100.29%	100.28%	100.29%	100.29%	100.29%
Frequency scalability	100.00%	99.90%	99.87%	99.86%	99.68%	99.40%	99.26%	98.93%	98.44%	97.97%

Table 2: Analysis done on Thu Dec 4 06:35:09 PM CET 2025, paco1114

Statistics about explicit tasks in parallel fraction										
Number of threads	1	2	4	8	12	16	20	24	28	32
Number of explicit tasks executed (total)	13345.0	13345.0	13345.0	13345.0	13345.0	13345.0	13345.0	13345.0	13345.0	13345.0
LB (number of explicit tasks executed)	1.0	0.97	0.53	0.39	0.6	0.51	0.49	0.55	0.38	0.51
LB (time executing explicit tasks)	1.0	0.99	0.94	0.85	0.85	0.82	0.73	0.73	0.75	0.8
Time per explicit task (average us)	106.47	106.99	107.12	109.36	109.7	112.78	114.04	116.49	118.96	121.62
Overhead per explicit task (synch %)	0.11	1.83	5.06	16.06	26.32	36.74	47.62	55.91	67.72	75.32
Overhead per explicit task (sched %)	0.21	0.26	0.37	0.58	1.4	3.1	4.36	5.9	7.5	9.31
Number of taskwait/taskgroup (total)	4041.0	4041.0	4041.0	4041.0	4041.0	4041.0	4041.0	4041.0	4041.0	4041.0

Table 3: Analysis done on Thu Dec 4 06:35:09 PM CET 2025, paco1114

Paraver analysis

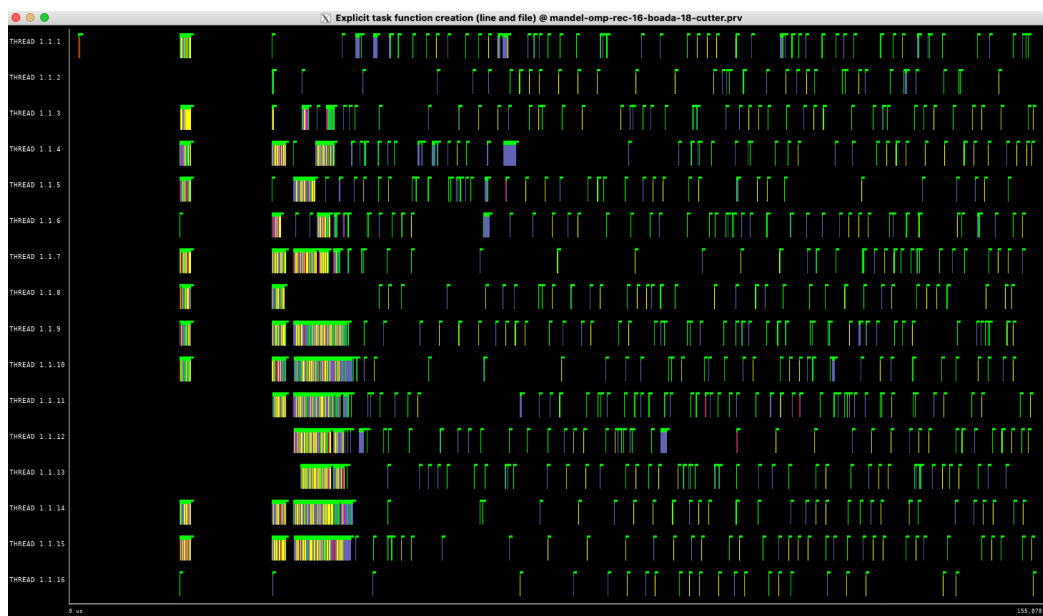


Aquesta vista mostra una distribució de feina molt més saludable que la versió *Leaf*. Els fils presenten blocs de color blau (còmput) significatius, indicant que l'estratègia d'arbre reparteix bé la feina. Tot i això, s'observa un cert 'efecte escala' amb zones vermelles/grogues, la qual cosa indica que encara hi ha un cert **desequilibri de càrrega** quan es tanquen les darreres branques de l'arbre i uns fils acaben abans que els altres.



El paral·lelisme es manté alt durant gran part de l'execució, oscil·lant prop del màxim. A diferència de la versió iterativa *Fine Grain* (que era plana), aquí hi ha més variabilitat degut a la naturalesa dinàmica de l'arbre recursiu. La caiguda progressiva al final confirma el **desequilibri de càrrega** esmentat anteriorment.

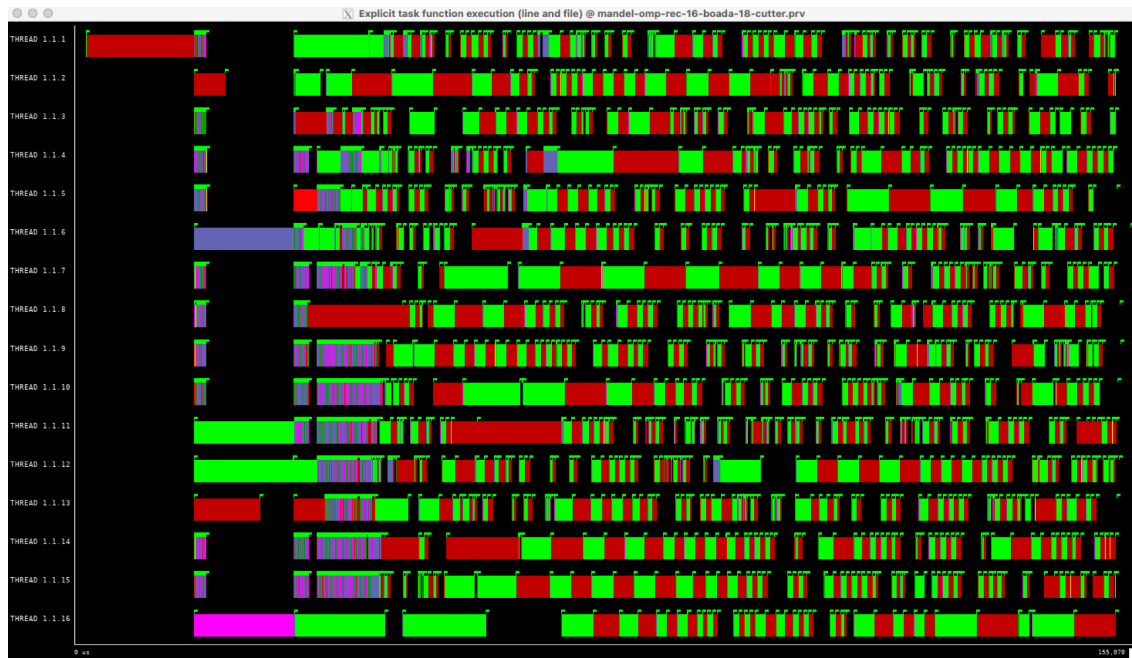
- Creation



La creació de tasques (marques verdes) apareix **distribuïda en el temps i en l'espai** (diferents fils). No és un bloc massiu inicial com a l'estratègia *Original Tile* o *Leaf*, sinó que

les tasques es van creant progressivament a mesura que els fils processen les branques i baixen per l'arbre. Això ajuda a evitar colls d'ampolla inicials.

- Execution



Aquesta vista confirma que l'estratègia *Tree* genera tasques de mides molt variades (els blocs verds tenen longituds diferents). A diferència de l'estratègia *Leaf* (on les tasques eren punts invisibles) o l'*Original Tile* (on eren blocs gegants), aquí tenim una **granularitat intermèdia i dinàmica**. S'observa com els fils van processant les diferents branques de l'arbre, tot i que els espais vermells/liles intercalats mostren l'overhead de sincronització (taskwait) necessari quan una tasca pare ha d'esperar que acabin els seus fills.

Strong scalability test

Shell

```
paco1114@boada-11:~/lab4$ cat speedup-partial-mandel-omp-rec-boada-20.txt
```

```
1    .99482740483761743944
4    3.80987301507368986470
8    7.08059553659516224004
12   9.86003486231557454942
16   12.17508063056116188471
20   14.03276040620121489404
24   15.77586731191951136867
28   17.31632302733220164412
32   18.5455232708256730808
```

```
paco1114@boada-11:~/lab4$ cat elapsed-partial-mandel-omp-rec-boada-20.txt
```

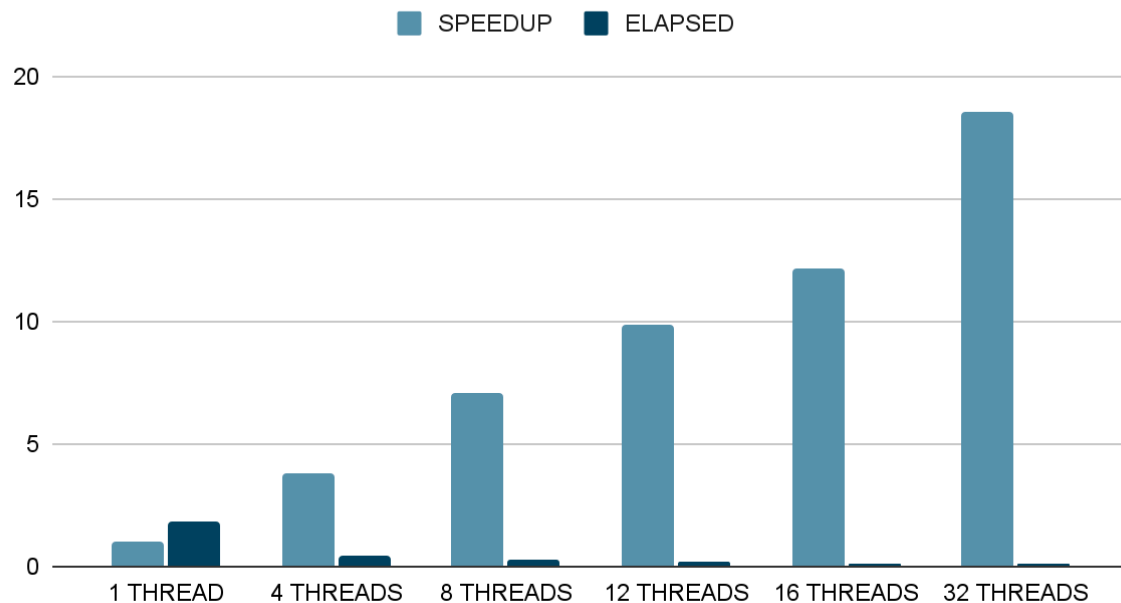
```
1    1.825196
4    0.476592
8    0.256441
12   0.184153
```

```

16  0.149137
20  0.129394
24  0.115097
28  0.104858
32  0.097908

```

Punts obtinguts



Leaf+Tree strategy

3.2.Execution + 3.3.Correctness

Shell

```

paco1114@boada-11:~/lab4$ make clean
rm -rf mandel-seq-iter mandel-seq-rec mandel-omp-iter mandel-omp-rec logs
*.log
paco1114@boada-11:~/lab4$ make mandel-omp-rec
icx -I. -I/Soft/stb/20200430 -o mandel-omp-rec -g -O3 -fp-model precise
-std=c99 -qopenmp mandel-omp-rec.c -lX11 -L/usr/X11R6/lib -lm
paco1114@boada-11:~/lab4$ sbatch submit-omp.sh mandel-omp-rec 20
Submitted batch job 367651
paco1114@boada-11:~/lab4$ squeue -u $USER
          JOBID PARTITION    NAME    USER  ST       TIME  NODES
NODELIST(REASON)
    367636 interacti _interac paco1114  R       8:29      1 boada-11

```



```
paco1114@boada-11:~/lab4$ cmp mandel_histogram.out
mandel_histogram_seq_rec.out
paco1114@boada-11:~/lab4$ cmp mandel_image.jpg mandel_image_seq_rec.jpg
```

2.4. Performance analysis

Aquesta estratègia combinada obté un bon resultat, amb un Speedup de 15.45 i un temps d'execució de 0.12 segons. Tot i ser molt superior a la versió Leaf original, queda lleugerament per darrere de l'estratègia Tree pura (que arribava a 17.72).

Aquests resultats suggereixen que, per a aquest tipus de problema, intentar paral·lelitzar tant a nivell d'arbre com a nivell de fulla introdueix un petit overhead addicional o desequilibri que no compensa davant la simplicitat i eficiència de l'estratègia Tree sola.

Modelfactor analysis

Overview of whole program execution metrics										
Number of threads	1	2	4	8	12	16	20	24	28	32
Elapsed time (sec)	1.84	0.93	0.49	0.27	0.19	0.16	0.14	0.14	0.12	0.12
Speedup	1.00	1.98	3.79	6.91	9.46	11.30	13.02	13.57	14.75	15.45
Efficiency	1.00	0.99	0.95	0.86	0.79	0.71	0.65	0.57	0.53	0.48

Table 1: Analysis done on Wed Dec 3 01:40:08 PM CET 2025, paco1114

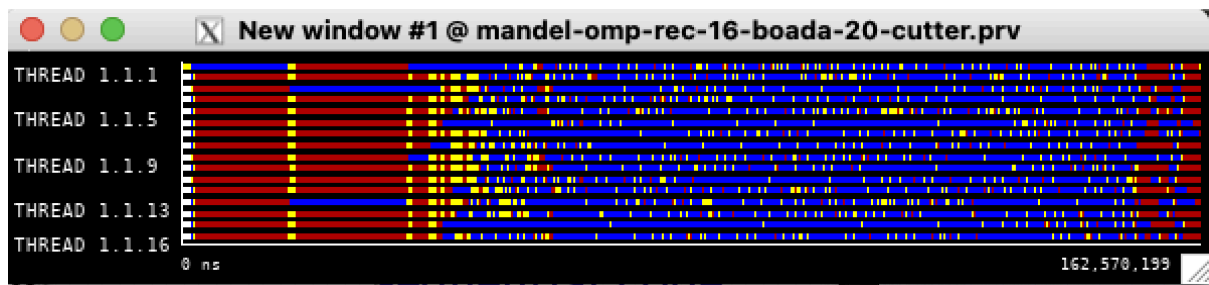
Overview of the Efficiency metrics in parallel fraction, $\phi=99.98\%$										
Number of threads	1	2	4	8	12	16	20	24	28	32
Global efficiency	99.91%	98.73%	94.62%	86.41%	78.85%	70.67%	65.22%	56.62%	52.72%	48.33%
Parallelization strategy efficiency	99.91%	98.74%	94.71%	86.66%	79.16%	71.16%	65.84%	57.45%	53.71%	49.56%
Load balancing	100.00%	99.15%	96.12%	93.98%	88.83%	85.88%	84.23%	78.08%	77.79%	75.27%
In execution efficiency	99.91%	99.59%	98.53%	92.21%	89.12%	82.86%	78.17%	73.59%	69.04%	65.84%
Scalability for computation tasks	100.00%	100.00%	99.90%	99.71%	99.61%	99.32%	99.07%	98.55%	98.16%	97.52%
IPC scalability	100.00%	99.87%	99.86%	99.78%	99.74%	99.68%	99.67%	99.61%	99.58%	99.57%
Instruction scalability	100.00%	100.15%	100.15%	100.15%	100.15%	100.15%	100.15%	100.15%	100.15%	100.15%
Frequency scalability	100.00%	99.97%	99.90%	99.78%	99.71%	99.49%	99.24%	98.79%	98.43%	97.79%

Table 2: Analysis done on Wed Dec 3 01:40:08 PM CET 2025, paco1114

Statistics about explicit tasks in parallel fraction										
Number of threads	1	2	4	8	12	16	20	24	28	32
Number of explicit tasks executed (total)	7071.0	7071.0	7071.0	7071.0	7071.0	7071.0	7071.0	7071.0	7071.0	7071.0
LB (number of explicit tasks executed)	1.0	0.96	0.56	0.53	0.53	0.65	0.52	0.55	0.69	0.75
LB (time executing explicit tasks)	1.0	1.0	0.98	0.93	0.89	0.87	0.84	0.78	0.78	0.75
Time per explicit task (average us)	256.98	257.57	257.79	258.62	259.77	261.42	263.33	265.54	267.84	270.04
Overhead per explicit task (synch %)	0.0	1.16	5.44	14.77	24.88	38.27	48.37	68.74	79.12	93.62
Overhead per explicit task (sched %)	0.09	0.12	0.14	0.4	0.81	1.33	2.0	2.75	3.53	4.33
Number of taskwait/taskgroup (total)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

Table 3: Analysis done on Wed Dec 3 01:40:08 PM CET 2025, paco1114

Paraver analysis

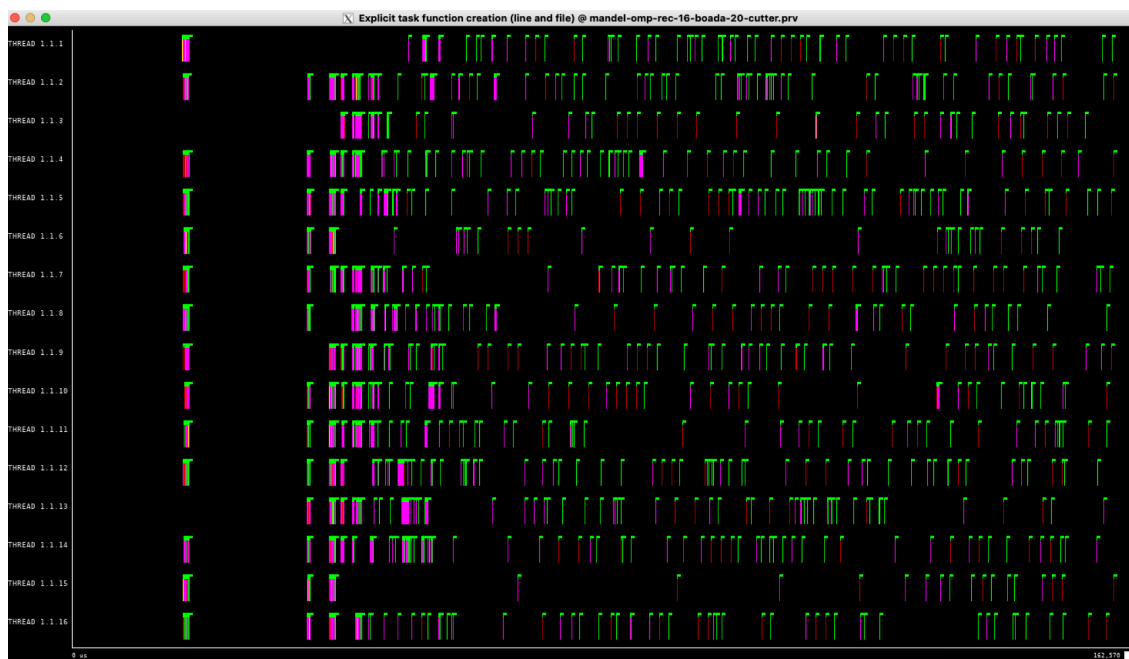


En aquesta vista s'observa un comportament híbrid. Veiem blocs de còmput (blau), però també una quantitat significativa de color vermell (sincronització/overhead) intercalada, especialment a la primera meitat. Això indica que la gestió de les tasques de les fulles introdueix una penalització que no teníem a l'estratègia *Tree* pura, fent que els fils passin temps esperant o gestionant la recursivitat.



La gràfica de paral·lelisme és força irregular i presenta molts pics i valls ('dents de serra'). Tot i que arriba a valors alts, no es manté constant al màxim com en les versions iteratives. Això reflecteix la naturalesa dinàmica de l'execució híbrida, on la generació i execució de tasques varia constantment.

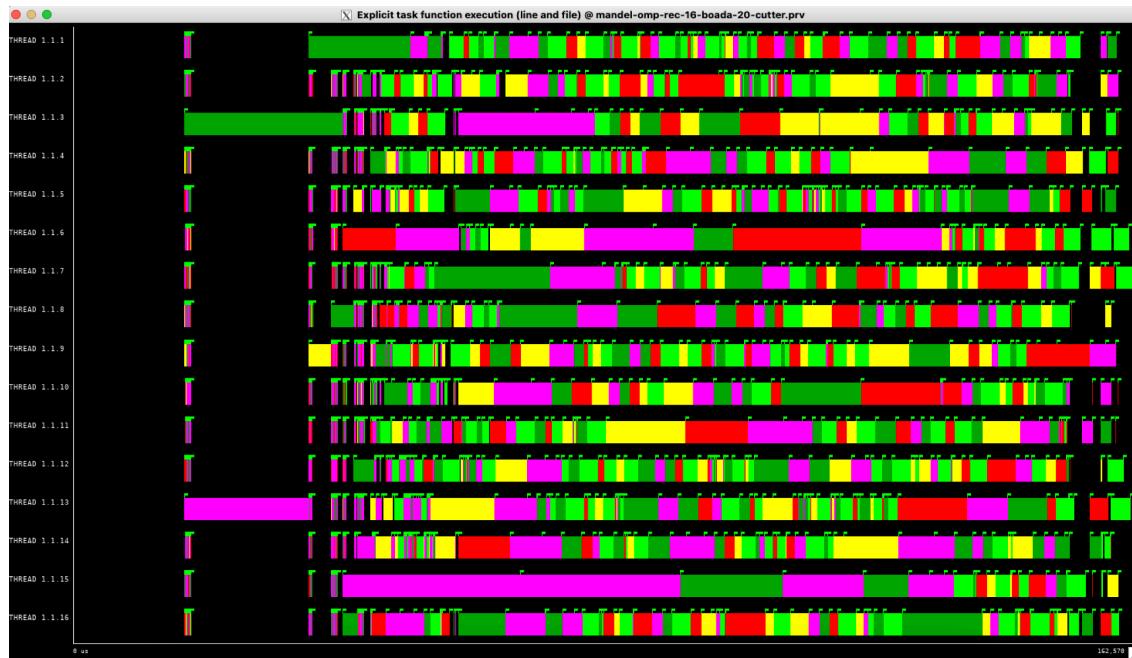
- Creation



En aquesta gràfica veiem clarament que la creació de tasques funciona per **ràfegues (bursts)**. No és contínua ni massiva al principi, sinó que apareixen grups de línies verdes

verticals separats per espais buits negres. Això indica que els fils s'aturen a crear noves tasques només en moments concrets (quan subdivideixen una branca de l'arbre), i després es dediquen a executar-les abans de tornar a crear-ne més.

- Execution



En aquesta vista observem **barres de longitud considerable i colors variats** (verd, groc, rosa, vermell). Això és molt positiu, ja que confirma que l'estratègia híbrida (Leaf+Tree) aconsegueix generar tasques amb prou càrrega computacional (bona granularitat), evitant els 'punts microscòpics' de la versió *Leaf* original. La variabilitat en la llargada d'aquestes barres reflecteix la naturalesa irregular del Mandelbrot..

Strong scalability test

Shell

```
paco1114@boada-11:~/lab4$ cat speedup-partial-mandel-omp-rec-boada-19.txt
```

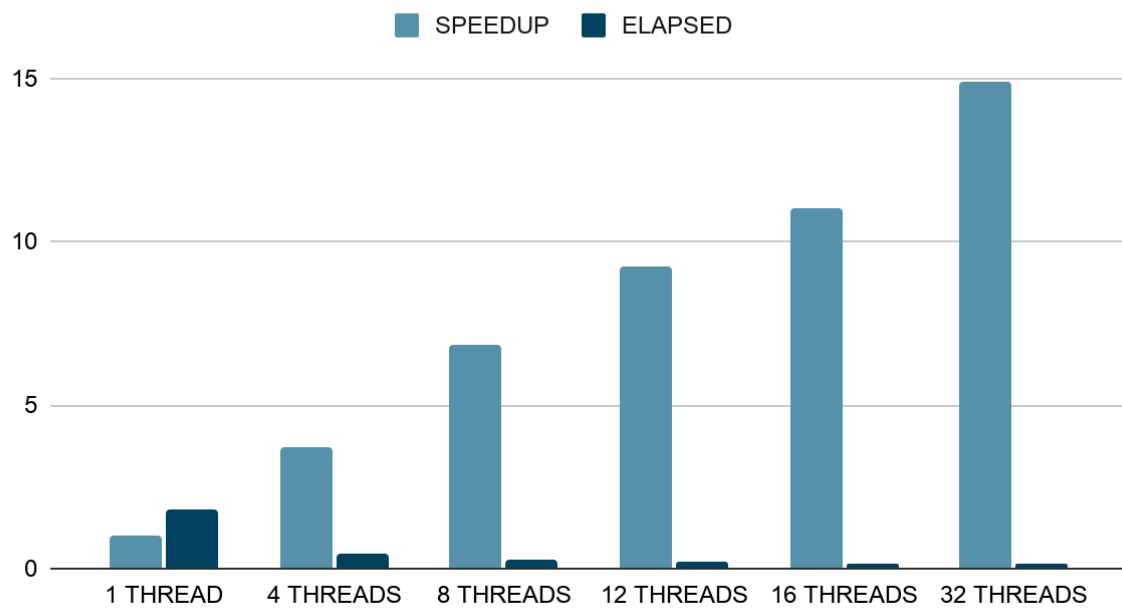
```
1    .99284314626006552157
4    3.74099169161106109642
8    6.83951714656974444925
12   9.25142903722119764017
16   11.03510594984169811435
20   12.67573642328633254223
24   13.91354250469294717082
28   14.58938370196595136138
32   14.90687747295145216634
```

```
paco1114@boada-11:~/lab4$ cat elapsed-partial-mandel-omp-rec-boada-19.txt
```

```
1    1.829016
4    0.485413
8    0.265505
12   0.196286
```

16 0.164559
20 0.143260
24 0.130515
28 0.124469
32 0.121818

Punts obtinguts



A.2 Final comparison between all the parallel strategies

Table A.2: Summary of the elapsed execution time

Version	1 Thread	4 Threads	8 Threads	12 Threads	16 Threads	32 Threads
Iterative: Tile	3.54	1.34	0.75	0.74	0.74	0.74
Iterative: Fine Grain	3.54	0.89	0.45	0.37	0.28	0.15
Recursive: Leaf	1.83	1.42	1.42	1.42	1.42	1.42
Recursive: Improved Leaf	1.83	0.53	0.40	0.40	0.40	0.40
Recursive: Tree	1.83	0.48	0.26	0.18	0.15	0.10

Best Parallel Implementation

Versió: Recursive: Tree

Com podem observar a la Taula A.2 actualitzada, la millor implementació és la Recursive: Tree. Amb 32 fils, aconsegueix un temps d'execució absolut rècord de 0.10 segons, superant la versió Iterative: Fine Grain (0.15s). Això demostra que dividir l'espai de treball seguint les branques de l'arbre de recursió s'adapta millor a la naturalesa fractal del Mandelbrot, aconseguint minimitzar l'overhead i maximitzar l'eficiència respecte a totes les altres estratègies.

Conclusió Final

En aquest laboratori hem dissenyat, implementat i analitzat diverses estratègies de descomposició de tasques per al càlcul del conjunt de Mandelbrot utilitzant OpenMP, comprenent l'impacte de la granularitat, l'overhead de gestió de tasques i el balanceig de càrrega en el rendiment paral·lel.

Pel que fa a les estratègies iteratives, s'ha demostrat que la descomposició *Original Tile* patia d'un fort desequilibri de càrrega (efecte escala al final de l'execució). L'evolució cap a l'estratègia *Fine Grain* ha estat determinant: en reduir la granularitat de les tasques (augmentant-ne el nombre però fent-les més fines), s'ha aconseguit un balanceig de càrrega gairebé perfecte, mantenint tots els fils ocupats fins al final de l'execució.

Quant a les estratègies recursives, hem comprovat experimentalment com l'estratègia *Leaf* pura és inviable a causa de l'elevat overhead de creació i sincronització de milions de tasques petites. La introducció del mecanisme de *Cutoff (Improved Leaf)* ha solucionat aquest problema d'overhead, però no ha pogut resoldre totalment el desequilibri de càrrega. Finalment, l'estratègia *Tree* ha demostrat ser la més robusta, gràcies a una distribució estructural que s'adapta millor a la naturalesa del problema.

Basant-nos en els resultats finals recollits a la Taula A.2, la millor implementació és la **Recursive: Tree**. Aquesta versió ha aconseguit el temps d'execució mínim absolut (**0.10s** amb 32 fils), superant la versió *Iterative: Fine Grain* (0.15s). Això demostra que, tot i que l'estratègia iterativa de gra fi és excel·lent per balancejar la càrrega, l'estratègia recursiva d'arbre és capaç d'aprofitar millor l'estructura irregular del fractal per minimitzar el temps total de còmput.